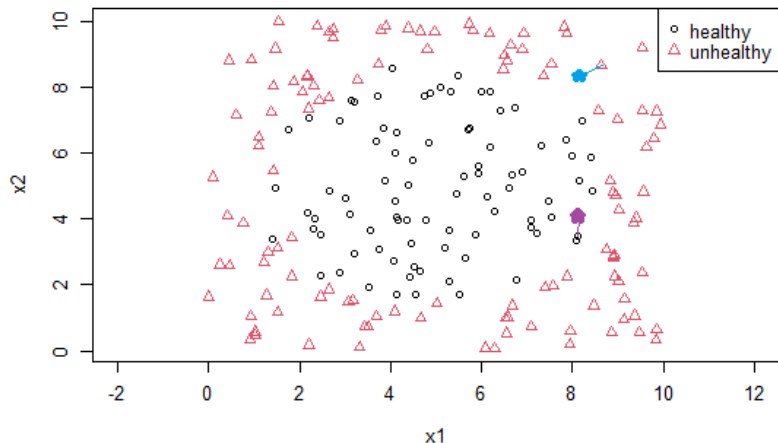


Transformations, scaling, and pipelines

Albert Y. Kim

Coming up later: k -nearest neighbors (kNN)

A simple rule is to classify each new data-point using its *nearest neighbors*, or the observation closest to it's x_2 and x_1 coordinates:



Scaling

Because kNN relies upon distance calculations, rescaling the predictors is important:

1. **Standardization:**

$$x_i^* = \frac{x_i - \text{mean}(x)}{\text{sd}(x)}$$

2. **Robust scaling:**

$$x_i^* = \frac{x_i - \text{median}(x)}{\text{IQR}(x)}$$

3. **Min-Max scaling:**

$$x_i^* = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

4. **Max-Absolute scaling:**

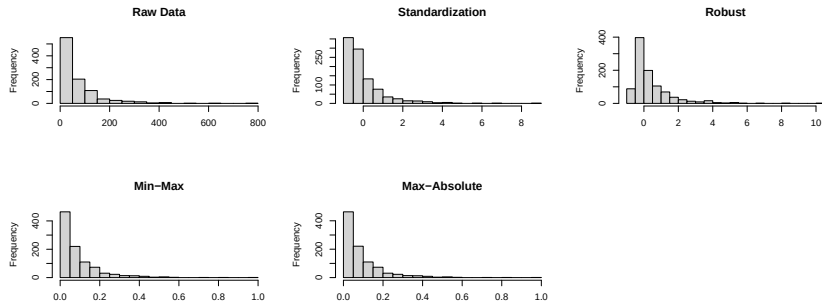
$$x_i^* = \frac{x_i}{\max(|x|)}$$

Standardization

- ▶ Standardization on the other hand forces features to have a *mean of zero* and a *standard deviation of one*
- ▶ Robust scaling forces features to have a *median of zero*, and it can be beneficial for data with large outliers
- ▶ Min-Max scaling maps each feature onto a $[0,1]$ interval, which can have computational advantages
- ▶ Max-Absolute scaling is similar to Min-Max scaling, but the output range is $[-1,1]$ and it will *preserve exact zeros* (important for sparse data)

Scaling vs. Normalization

Scaling changes the range of your data, it does not change the distributional shape:

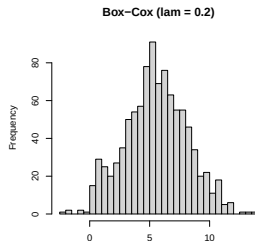
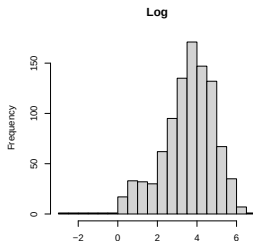
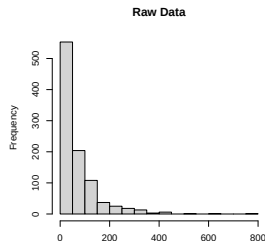


However, the choice of scaler does matter for k -nearest neighbors, since dimensions are rescaled individually

Normalization

If you'd like to change the distributional shape of your data to reduce the effects of skew/outliers, two strategies are:

1. Log-transformation - simply taking the logarithm of each of the variable's values
2. Box-Cox transformation - $x_i^* = \frac{x_i^\lambda - 1}{\lambda}$ for $\lambda \neq 0$ and $X > 0$



Putting it all together

In today's lab you'll:

1. Split the data into training and validation sets
2. Set up a data preparation *pipeline* (ie: rescaling, normalization, dimension reduction, etc.)